Okta & Amazon Cognito

Federating an External Identity Provider into the Cognito Ecosystem
Architecture, SAML and OIDC Setup, Runtime Flow, and Code Examples

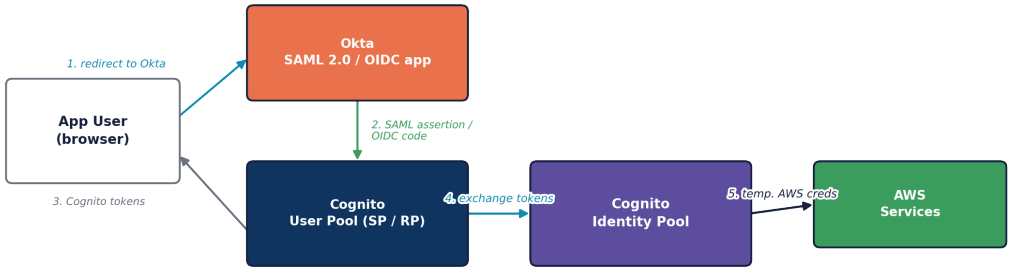
---

This reference explains how Okta fits into the Amazon Cognito ecosystem as an external identity provider, covers both supported federation protocols (SAML 2.0 and OIDC), walks through the runtime sign-in sequence in detail, and provides working configuration and application code for each stage — from Terraform/boto3 provider setup through token exchange with a Cognito Identity Pool for temporary AWS credentials.

> All explanations and code samples are original, written to accompany AWS's and Okta's public documentation rather than reproduced from any third-party source.

1. Where Okta Fits in the Ecosystem

Okta does not replace Cognito, and it doesn't sit downstream of it either — it federates into the Cognito User Pool as an external identity provider, in the same architectural slot that a generic SAML or OIDC provider occupies. Cognito still issues its own tokens after the handoff, so everything downstream of sign-in — token validation, refresh, API Gateway authorizers, and Identity Pool credential exchange — works exactly the same as it does for native username/password sign-in.



Okta federates into the Cognito User Pool as an external IdP (SAML 2.0 or OIDC). Cognito issues its own tokens after federation, so everything downstream — token validation, refresh, and Identity Pool exchange — works exactly as it does for native sign-in.

Okta federates into the Cognito User Pool; the pool issues its own tokens which flow unchanged into the rest of the architecture, including the Identity Pool credential exchange.

Two federation protocols, compared

	SAML 2.0	OIDC
Protocol fit	Classic enterprise SSO standard	Modern OAuth 2.0-based standard
Okta app type	SAML 2.0 app integration	OIDC (Web App) app integration
Cognito identifies the user by	The NameID claim in the SAML assertion	The sub claim (Cognito auto-maps this to username)
Typical choice	Enterprises already standardized on SAML	New integrations wanting standard OAuth2 semantics

2. Path A: Okta as a SAML 2.0 Identity Provider

On the Okta side

Create a SAML 2.0 app integration in Okta and point it at Cognito's endpoint. The Assertion Consumer Service (ACS) URL is:

```
https://<cognitoDomain>.auth.<region>.amazoncognito.com/saml2/idpresponse
```

and the Audience URI (SP Entity ID) is:

```
urn:amazon:cognito:sp:<userPoolId>
```

If you're using a custom domain rather than a Cognito-provided one, substitute your custom domain directly: https://your-custom-domain/saml2/idpresponse. You'll find your domain prefix and region in the Domain menu of your user pool, and your user pool ID on the Overview dashboard.

On the Cognito side

In the Cognito console, open your user pool, go to Authentication → Social and external providers, choose Add identity provider → SAML, name it (e.g. “Okta”), and provide the metadata document either by uploading the file or entering the metadata endpoint URL copied from Okta's Sign On tab. Map the SAML email attribute to your user pool's email attribute.

Terraform

```
resource "aws_cognito_identity_provider" "okta_saml" {
  user_pool_id  = aws_cognito_user_pool.main.id
  provider_name = "Okta"
  provider_type = "SAML"

  provider_details = {
    MetadataURL = "https://your-org.okta.com/app/exkXXXXXXXXXXXX/sso/saml/metadata"
  }

  attribute_mapping = {
    email    = "email"
    username = "sub"
  }
}

resource "aws_cognito_user_pool_client" "app" {
  # ...
  supported_identity_providers = ["COGNITO", aws_cognito_identity_provider.okta_saml.provider_name]
  callback_urls                = ["https://myapp.example.com/callback"]
  allowed_oauth_flows          = ["code"]
  allowed_oauth_scopes         = ["openid", "email", "profile"]
  allowed_oauth_flows_user_pool_client = true
}
```

boto3

```
import boto3
client = boto3.client("cognito-idp")

client.create_identity_provider(
    UserPoolId=USER_POOL_ID,
    ProviderName="Okta",
    ProviderType="SAML",
    ProviderDetails={
        "MetadataURL": "https://your-org.okta.com/app/exkXXXXXX/sso/saml/metadata",
    },
    AttributeMapping={"email": "email"},
)
```

Gotcha to flag: Cognito identifies a SAML-federated user by the NameID claim in their assertion, regardless of the user pool's case-sensitivity settings. If NameID is mapped from a mutable attribute like email and the user later changes their email address, they can no longer sign in under the same identity. Map NameID from something immutable on the Okta side (an internal user GUID), not email.

3. Path B: Okta as an OIDC Identity Provider

On the Okta side

Create an OIDC app integration and note the Issuer URL from the Sign On page under OpenID Connect ID Token — typically https://your-org.okta.com/oauth2/default, or a custom authorization server's issuer URL.

On the Cognito side

Open your user pool, go to Federation → Identity providers → OpenID Connect, enter a provider name, the Client ID and Client secret from Okta, keep Attributes request method as GET, and enter your Authorize scope values separated by spaces. The openid scope is required for OIDC IdPs; typically add profile and email alongside it.

Terraform

```
resource "aws_cognito_identity_provider" "okta_oidc" {
  user_pool_id  = aws_cognito_user_pool.main.id
  provider_name = "Okta"
  provider_type = "OIDC"

  provider_details = {
    client_id                = var.okta_client_id
    client_secret            = var.okta_client_secret
    attributes_request_method = "GET"
    oidc_issuer              = "https://your-org.okta.com/oauth2/default"
    authorize_scopes         = "openid profile email"
  }

  attribute_mapping = {
    email    = "email"
    username = "sub"
  }
}
```

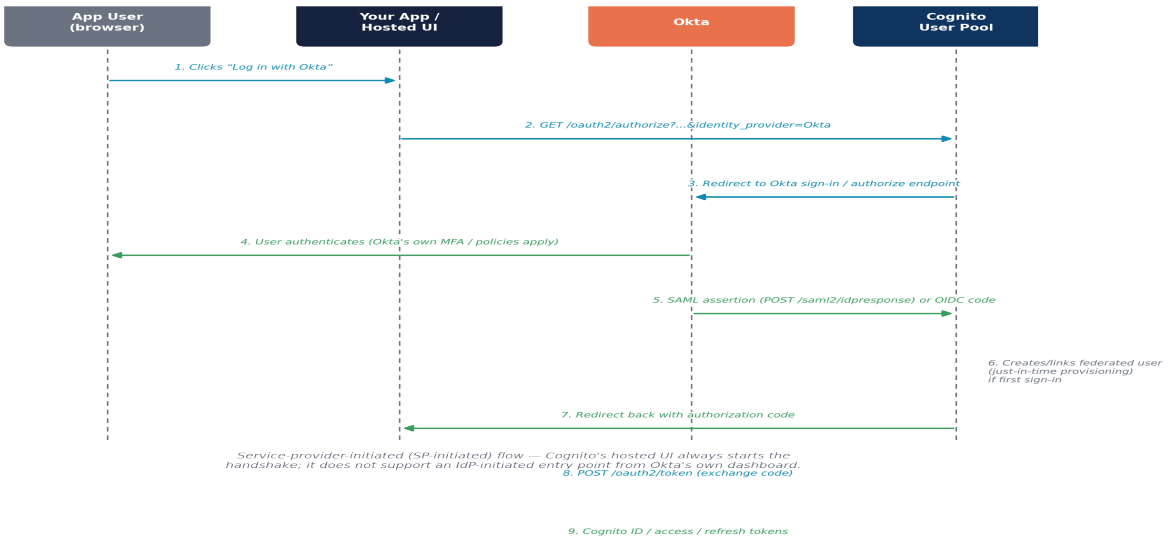
boto3

```
client.create_identity_provider(
    UserPoolId=USER_POOL_ID,
    ProviderName="Okta",
    ProviderType="OIDC",
    ProviderDetails={
        "client_id": OKTA_CLIENT_ID,
        "client_secret": OKTA_CLIENT_SECRET,
        "attributes_request_method": "GET",
        "oidc_issuer": "https://your-org.okta.com/oauth2/default",
        "authorize_scopes": "openid profile email",
    },
    AttributeMapping={"email": "email", "username": "sub"},
)

client.update_user_pool_client(
    UserPoolId=USER_POOL_ID,
    ClientId=APP_CLIENT_ID,
    SupportedIdentityProviders=["COGNITO", "Okta"],
    CallbackURLs=["https://myapp.example.com/callback"],
    AllowedOAuthFlows=["code"],
    AllowedOAuthScopes=["openid", "email", "profile"],
    AllowedOAuthFlowsUserPoolClient=True,
)
```

Cognito automatically maps the OIDC sub claim to the username in the destination user profile; every other attribute (email, name, custom claims) needs an explicit mapping rule.

4. Runtime Flow, End to End



The full SP-initiated redirect sequence, from the user's click through to Cognito tokens landing back in the app.

1. User clicks “Log in with Okta” in the app, which redirects to Cognito's Hosted UI authorize endpoint with an identity_provider parameter naming the configured IdP.
2. Cognito redirects the browser to Okta's sign-in page (the SAML SSO URL or the OIDC authorize endpoint).
3. The user authenticates at Okta, subject to Okta's own policies — its own MFA, device trust, etc. This centralization is often the whole point of the integration.
4. Okta redirects back with a SAML assertion (POSTed to /saml2/idpresponse) or an OIDC authorization code.
5. Cognito creates a user profile in the local user pool for this Okta user if one doesn't exist yet (just-in-time provisioning), and redirects back to the client app with an authorization code.
6. The client app makes a backend call to Cognito's token endpoint to exchange that code for ID and access tokens, then validates the ID token as usual.

Step 1: the Hosted UI authorize request

```
https://myapp-domain.auth.us-east-1.amazoncognito.com/oauth2/authorize
  ?client_id=4c4s2s125or70hbd88ggcb36dv
  &response_type=code
  &scope=email+openid+phone+profile
  &redirect_uri=https://myapp.example.com/callback
  &identity_provider=Okta
```

Step 6: exchanging the code for tokens

```
const tokenResponse = await fetch(
  'https://myapp-domain.auth.us-east-1.amazoncognito.com/oauth2/token',
  {
    method: 'POST',
    headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
    body: new URLSearchParams({
      grant_type: 'authorization_code',
      client_id: APP_CLIENT_ID,
      code: authorizationCode, // from ?code= on your redirect_uri
      redirect_uri: 'https://myapp.example.com/callback',
    }),
  }
);
const { id_token, access_token, refresh_token } = await tokenResponse.json();
```

From this point on these are ordinary Cognito tokens: validate them against the same JWKS endpoint, refresh them the same way, and exchange them with an Identity Pool for AWS credentials the same way — the pool doesn't distinguish federated users from SRP-authenticated ones.

Amplify: wrapping the redirect

```
import { signInWithRedirect } from 'aws-amplify/auth';

await signInWithRedirect({
  provider: { custom: 'Okta' }, // must match the ProviderName configured in Cognito
});
```

5. Attribute Mapping and Group Claims

Group or role information from Okta commonly needs to ride along as a custom claim so your app's authorization logic can use it. A typical pattern: map an appgroup OIDC attribute (or a SAML group attribute statement) to a custom:groups user pool attribute, then use a Pre Token Generation Lambda trigger to reshape that into whatever claim format your app expects. In multi-tenant setups, additional mapped attributes are often used to identify the tenant a federated user belongs to.

6. A Limitation Worth Flagging

Cognito's hosted UI federation is SP-initiated only: the flow always starts at your app or Cognito's hosted UI and redirects out to Okta. Cognito does not support an IdP-initiated flow — a user can't start at Okta's own dashboard, click your app's tile, and land already signed in. They need to go through your app's sign-in entry point first. This is worth designing around if your organization relies on Okta's dashboard as the primary launch point for internal tools.

7. Alternate Architecture: Okta Feeding an Identity Pool Directly

A second, less common pattern is worth knowing about. Instead of federating Okta into a Cognito User Pool, you can register Okta as an OIDC provider directly on an Identity Pool (via an IAM OIDC identity provider ARN). The app authenticates against Okta directly, obtains an Okta-issued ID token, and hands that straight to GetId / GetCredentialsForIdentity with the Logins map keyed by Okta's issuer URL — skipping the Cognito User Pool, and any Cognito-issued tokens, entirely.

This trades away Cognito's unified token format and hosted UI for a simpler, more direct path when the only requirement is temporary AWS credentials rather than an application-level session managed by Cognito.

This document is an original technical write-up prepared to accompany AWS's and Okta's public documentation. Endpoint paths, claim names, and configuration field names (ACS URL, SP entity ID, NameID, authorize scopes, and related terms) are drawn from AWS's and Okta's published reference material, described here in original wording.
Primary references: AWS re:Post, “Set up Okta as a SAML identity provider in an Amazon Cognito user pool” and “Set up Okta as an OIDC identity provider” — repost.aws/knowledge-center; AWS Documentation, “Using SAML identity providers with a user pool” and “Using OIDC identity providers with a user pool” — docs.aws.amazon.com/cognito/latest/developerguide/; Okta Developer Documentation — developer.okta.com